Moneyora

Personal Finance & AI Budget Planner

Plan. Track. Thrive.

SOFTWARE REQUIREMENTS SPECIFICATION

Document Version:	1.0 — Final
Date:	20 June 2026
Classification:	Industrial Grade
Architecture:	Offline-First Hybrid
Platform:	Flutter — iOS 14+ & Android 8.0+
Currency Default:	LKR (Sri Lanka Rupee)
Status:	APPROVED FOR DEVELOPMENT

Developed by Sanduni • Individual Developer Project

AI-Enhanced Personal Finance • Budget Planner & Receipt Scanner

TABLE OF CONTENTS

1. INTRODUCTION

1.1 Purpose

This Software Requirements Specification (SRS) defines the complete functional and non-functional requirements for **Moneyora** — an offline-first personal finance management mobile application. The document is intended to serve as the single authoritative reference for all design, development, testing, and evaluation activities throughout the project lifecycle.

Moneyora is a purpose-built personal finance application that introduces two major AI-powered capabilities: an intelligent **Money Plan Generator** that builds personalized budget plans from historical spending data, and an **OCR-based Receipt Scanner** that automatically categorizes scanned bills and receipts.

1.2 Scope

The Moneyora application shall provide:

- Complete expense and income tracking with multi-account support
- Intuitive category-based transaction management
- Visual analytics — donut/pie charts, trend lines, date filtering
- AI-driven Money Plan generation based on user spending history and patterns
- OCR-powered receipt/bill scanning with automatic item categorization
- Fully offline-first operation — 100% functional without internet
- Optional cloud backup/sync (Google Drive, Dropbox)
- Industrial-grade data security with AES-256 encryption at rest

1.3 Intended Audience

Audience	Relevant Sections
Developer (Sanduni)	Full document — all sections
Evaluators / Academic Panel	Sections 1, 2, 3, 5, 8
Future Contributors	Sections 3, 6, 7, 4
Testers / QA	Sections 3, 5

1.4 Definitions & Acronyms

Term / Acronym	Definition
SRS	Software Requirements Specification
FR	Functional Requirement
NFR	Non-Functional Requirement
OCR	Optical Character Recognition — extracting text from images

ML Kit	Google's on-device Machine Learning SDK
LKR	Sri Lanka Rupee — default currency
Money Plan	AI-generated budget allocation for a user-selected time period
Fixed Expense	Recurring expense with predictable amount (e.g. rent, subscriptions)
Variable Expense	Non-recurring expense with fluctuating amount (e.g. food, taxi)
Seasonal Expense	Periodic spending spike tied to events or seasons
TFLite	TensorFlow Lite — on-device neural network inference runtime
DAO	Data Access Object — database abstraction layer pattern
SQLCipher	AES-256 encrypted SQLite extension
Receipt Batch	Group of transactions created from a single receipt scan
Confidence Level	System's certainty score (High/Medium/Low) for a prediction

1.5 Document Conventions

Convention	Meaning
FR-EXP-001	Functional Requirement — Expense Tracking, item 1
FR-PLN-001	Functional Requirement — Money Plan, item 1
FR-RCP-001	Functional Requirement — Receipt Scanner, item 1
NFR-PER-001	Non-Functional Requirement — Performance, item 1
Bold text	Key terms, feature names, or critical requirements
SHALL	Mandatory requirement
SHOULD	Recommended but not mandatory
MAY	Optional feature

2. OVERALL DESCRIPTION

2.1 Product Perspective

Moneyora is a standalone personal finance mobile application built from the ground up, targeting iOS (14+) and Android (8.0+) via a single Flutter codebase. It features a visual donut chart home screen, a custom numeric keypad for fast expense entry, and multi-account and category management panels — alongside two AI-powered capabilities that differentiate it from conventional finance applications.

The product operates in an **offline-first hybrid model**: all core functionality — transaction recording, analytics, budget planning from local data — works without any internet connection. AI-enhanced features (cloud-based plan refinement, exchange rates) degrade gracefully when offline, falling back to on-device algorithms. The app is designed from the ground up as an independent product with its own architecture and feature set.

2.2 User Classes & Characteristics

User Class	Description	Scope of Access
Primary User	Individual managing personal daily finances	Full access to all features — transactions, plans, scanner, reports
Casual User	Occasional expense logging, minimal AI usage	Core transaction + chart features; may not use receipt scanner
Power User	Detailed budgeting and financial analysis	Heavy use of Money Plan, analytics, data export, multi-account
Restore User	Device migration or backup restore	Read-only access during restore validation

2.3 Operating Environment

Parameter	Specification
Mobile OS	iOS 14.0 or later / Android 8.0+ (API Level 26+)
Framework	Flutter (Dart) — single codebase cross-platform
Local Database	SQLite via sqflite + SQLCipher encryption
Minimum RAM	2 GB (3 GB recommended for ML features)
Storage	150 MB app base + user data + receipt images (configurable max 500 MB)
Camera	Required for receipt scanning feature
Internet	Optional — required only for cloud sync, AI API, exchange rates
Screen Size	4.7" to 6.7" phones; tablet support optional (Phase 2)
Biometric Sensors	Optional — fingerprint / Face ID for authentication

2.4 Design & Implementation Constraints

- The app **MUST** function 100% without internet connectivity for all core features
- All financial data **MUST** be encrypted at rest using AES-256 (SQLCipher)
- Total app size **MUST** remain under 80 MB (including on-device ML models)
- Receipt OCR must work offline using Google ML Kit Text Recognition v2
- The app must comply with Apple App Store and Google Play Store guidelines
- Flutter minimum SDK: Dart 3.0+ with null safety enforced
- Architecture: Clean Architecture with strict layer separation (Presentation → Domain → Data)
- AI API calls must be optional — AI features must degrade gracefully offline
- No user financial data shall be transmitted without explicit user consent

2.5 Assumptions & Dependencies

ID	Assumption / Dependency
A1	Device has at least 150 MB free storage for initial installation
A2	User has basic smartphone literacy (can use camera, navigate menus)
A3	Receipts are primarily in English or contain standard numeric formatting
A4	AI budget features improve accuracy after minimum 3 months of data
A5	Google ML Kit availability on target platforms remains unchanged
D1	Dependency: sqflite + sqflite_sqlcipher packages
D2	Dependency: google_mlkit_text_recognition package
D3	Dependency: fl_chart package for visualizations
D4	Dependency: OpenAI API or Gemini API (optional — for enhanced AI planning)
D5	Dependency: flutter_local_notifications for budget alerts

3. SYSTEM FEATURES & FUNCTIONAL REQUIREMENTS

3.1 Expense Tracking (FR-EXP)

All requirements in this section govern the recording, display, and management of expense transactions.

FR-EXP-001	The system **shall** allow users to record an expense with: amount, category, account, date, time, and optional text note.
FR-EXP-002	The system **shall** provide a custom numeric keypad supporting operators: +, -, *, / and equals for quick arithmetic during entry.
FR-EXP-003	The system **shall** include the following 15 default expense categories: **Bills, Car, Clothes, Communications, Eating Out, Entertainment, Food, Gifts, Health, House, Pets, Sports, Taxi, Toiletry, Transport**.
FR-EXP-004	The system **shall** allow users to create unlimited custom expense categories with user-defined name, icon (from built-in set), and color.
FR-EXP-005	The system **shall** support hierarchical sub-categories (Parent Category → Child Sub-category, maximum 2 levels deep).
FR-EXP-006	The system **shall** display all expenses in a chronological list, grouped by date with collapsible date section headers showing daily totals.
FR-EXP-007	The system **shall** show a running balance per account and a combined total balance across all included accounts.
FR-EXP-008	The system **shall** support recurring expenses with frequencies: Daily, Weekly, Monthly, Yearly, and Custom interval (N days).
FR-EXP-009	The system **shall** allow attaching a photo or scanned receipt image to any expense transaction for future reference.
FR-EXP-010	The system **shall** support splitting a single expense entry across multiple categories with individual amounts summing to the total.

3.2 Income Tracking (FR-INC)

FR-INC-001	The system **shall** allow users to record income with: amount, category, account, date, and optional note.
FR-INC-002	The system **shall** include 3 default income categories: **Deposits, Salary, Savings**.
FR-INC-003	The system **shall** allow creation of custom income categories.
FR-INC-004	The system **shall** support recurring income entries using the same frequency options as expenses.

FR-INC-005	The system **shall** visually distinguish income transactions from expenses — income displayed in green, expenses in red — in all list and chart views.

3.3 Account Management (FR-ACC)

FR-ACC-001	The system **shall** support multiple named accounts of types: Cash, Bank Account, Credit Card, Digital Wallet, Cryptocurrency, and Custom.
FR-ACC-002	Each account **shall** store: name, icon (from 20+ built-in icons), currency, initial balance, initial balance date, and an 'Include in Total Balance' toggle.
FR-ACC-003	The system **shall** display all accounts with current balances in a side-drawer panel accessible from the main screen.
FR-ACC-004	The system **shall** support account archiving — hiding accounts from active views without deleting transaction history.
FR-ACC-005	The system **shall** support multi-currency accounts with user-configurable exchange rates for balance conversion.
FR-ACC-006	The system **shall** provide 20+ built-in account icons including: Cash, AMEX, VISA, Mastercard, PayPal, Bitcoin, JCB, QIWI, Stripe, Discover, and others.

3.4 Transfer Management (FR-TRF)

FR-TRF-001	The system **shall** allow users to transfer funds between any two active accounts.
FR-TRF-002	Transfers **shall** be recorded as a single atomic transaction: debit from source account, credit to destination account.
FR-TRF-003	The system **shall** allow adding a note and custom date to each transfer.
FR-TRF-004	Transfers **shall** be visually distinct in transaction lists — shown with a bidirectional arrow icon and labeled 'From [Account]' / 'To [Account]'.

3.5 Money Plan Generator — Core AI Feature (FR-PLN)

Feature Overview

- The Money Plan Generator is the primary differentiating feature of Moneyora.
- It analyzes the user's historical transaction data to generate a personalized budget plan for any selected time period.
- The system uses statistical analysis (weighted moving averages, trend detection, seasonal adjustment) — working fully offline.
- Optional AI API calls can enhance predictions when internet is available.
- Users can review, adjust, and track actual spending against the generated plan in real time.

FR-PLN-001	The system **shall** provide a 'Create Money Plan' feature accessible from the main navigation menu.
FR-PLN-002	The user **shall** be able to select the plan period from: **Day, Week, Month, Year, Custom number of days** (e.g. 'Next 15 days'), or **Custom date range** (start date to end date).
FR-PLN-003	The system **shall** analyze the user's expense history over a configurable lookback period (default: last 6 months; user-adjustable from 1 to 24 months in Settings).
FR-PLN-004	The system **shall** automatically classify each category's expenses into: **Fixed** (recurring, low variance), **Variable** (fluctuating amounts), and **Seasonal** (periodic spikes).
FR-PLN-005	The system **shall** calculate and store per-category statistics: mean spend, median spend, standard deviation, min/max, transaction frequency, and trend direction (increasing / stable / decreasing).
FR-PLN-006	The system **shall** detect behavioral spending patterns including: weekday vs. weekend patterns, beginning-of-month vs. end-of-month patterns, seasonal cycles, and event-based spikes.
FR-PLN-007	Budget allocation **shall** be calculated using: exact recent average for fixed expenses; weighted moving average (recent 3 months: 60% weight, older months: 40%) for variable expenses; seasonal multiplier if historical data shows a spike in the planned period; and trend adjustment (+5-10% buffer if increasing trend, reduction if decreasing trend).
FR-PLN-008	The system **shall** offer two total budget modes: **Option A** — user sets a total available budget and the app distributes proportionally; **Option B** — app suggests a total based on historical income minus fixed expenses minus savings target.
FR-PLN-009	The system **shall** compute and display a daily allowance breakdown per category: e.g., 'You can spend Rs220 per day on Food this week.'
FR-PLN-010	The system **shall** display a confidence level (High / Medium / Low) for each category's allocation, based on data sufficiency and historical variance.

FR-PLN-011	The system **shall** allow users to manually adjust any category's generated allocation before saving, with all other allocations recalculating to maintain the total budget.
FR-PLN-012	The system **shall** provide a 'What-If' scenario tool: 'If I reduce [Category A] by X%, how much more can I allocate to [Category B]?'
FR-PLN-013	The system **shall** track actual spending vs. the active plan in real time, showing: percentage used per category, projected overspend/underspend, and color indicators (Green: on track, Yellow: 80-99%, Red: exceeded).
FR-PLN-014	When a category budget is exceeded, the system **shall** offer three responses: **Auto-Redistribute** (reduce remaining categories proportionally), **Manual Adjust** (user selects what to reduce), or **Carry Over** (deduct overspend from next period's allocation).
FR-PLN-015	The system **shall** allow saving multiple named plans (e.g., 'June Vacation Plan', 'Regular Monthly') and comparing any two plans side by side.

3.6 Receipt Scanner — Core AI Feature (FR-RCP)

Feature Overview

- The Receipt Scanner uses the device camera and Google ML Kit OCR to extract and parse bill items.
- Each identified line item is automatically matched to an expense category using keyword dictionaries and user history.
- The user reviews, edits, and confirms the scan before transactions are created.
- The system learns from user corrections to improve future categorization accuracy.
- All OCR processing happens on-device — no image is sent to any server without user consent.

FR-RCP-001	The system **shall** provide a 'Scan Receipt' button accessible from both the main screen and the add-expense flow.
FR-RCP-002	The system **shall** support image input via: device camera (live capture) or selection from photo gallery.
FR-RCP-003	The system **shall** perform offline image preprocessing: automatic deskew (correct tilt), contrast enhancement, binarization (black/white optimization), noise reduction, and perspective correction.
FR-RCP-004	The system **shall** perform OCR using **Google ML Kit Text Recognition v2** (offline-capable) to extract all text from the preprocessed receipt image.
FR-RCP-005	The system **shall** parse OCR output to identify: merchant/store name, receipt date and time, individual line items (product name, quantity, unit price, total price), receipt total, tax/VAT amount (if present), and receipt ID/number.
FR-RCP-006	The system **shall** handle multi-item receipts (e.g., grocery bills) and parse each line item individually.

FR-RCP-007	The system **shall** categorize each extracted item using a three-layer system: (1) **Keyword Dictionary** — pre-built map (e.g., 'Rice' → Food, 'Shampoo' → Toiletry, 'Petrol' → Car); (2) **User History** — if user previously categorized this merchant/item, reuse that mapping; (3) **Merchant Context** — if merchant is a pharmacy, bias toward Health. Each categorization receives a confidence score (0-100%).
FR-RCP-008	The system **shall** display a 'Review & Confirm' screen showing: receipt image thumbnail, parsed items list with suggested categories, editable fields for each item (name, amount, category), ability to merge or split items, and ability to discard incorrect items.
FR-RCP-009	The system **shall** create individual expense transactions for each confirmed item, linked under a single 'Receipt Batch' record for traceability.
FR-RCP-010	The system **shall** provide a 'Single Category' mode allowing the user to assign one category to the entire receipt total rather than per-item categorization.
FR-RCP-011	The system **shall** handle edge cases with explicit flags: handwritten receipts (lower accuracy warning), damaged/faded receipts ('Low Confidence' badge), and non-standard formats (utility bills, restaurant checks).
FR-RCP-012	The system **shall** store the original receipt image encrypted and linked to the resulting transaction(s) for future reference.
FR-RCP-013	The system **shall** provide a searchable 'Receipt History' view showing all previously scanned receipts with thumbnail, date, and total amount.
FR-RCP-014	The system **shall** allow re-scanning or re-processing a previously captured receipt image.
FR-RCP-015	The system **shall** implement learning from user corrections: when a user changes a suggested category, the system updates the personal keyword dictionary to use that mapping for future scans.

3.7 Analytics & Reporting (FR-RPT)

FR-RPT-001	The system **shall** display a donut/pie chart on the home screen showing expense distribution by category with percentage labels and category icons .
FR-RPT-002	The system **shall** support time period filters: **Day, Week, Month, Year, All, Custom Interval, Choose Date**.
FR-RPT-003	The system **shall** support account filter: All Accounts, or any specific single account.
FR-RPT-004	The system **shall** show income vs. expense comparison bar charts with net savings highlighted.
FR-RPT-005	The system **shall** provide trend line charts showing per-category spending over time.
FR-RPT-006	The system **shall** calculate and display: total income, total expenses, net savings, average daily spend, largest spending category, and month-over-month change.
FR-RPT-007	The system **shall** support data export to **CSV** and **PDF** formats.
FR-RPT-008	The system **shall** provide a transaction search function filtering by: amount range, category, note keyword, date range.
FR-RPT-009	The system **shall** show a calendar heatmap view highlighting daily spending intensity.

3.8 Settings & Configuration (FR-SET)

FR-SET-001	The system **shall** support Light Theme and Dark Theme with user toggle.
FR-SET-002	The system **shall** support multiple display languages (English default; framework ready for localization).
FR-SET-003	The system **shall** allow currency selection with LKR as default and support exchange rate configuration.
FR-SET-004	The system **shall** allow configuring first day of the week (Sunday/Monday) and first day of the month (1–28).
FR-SET-005	The system **shall** support passcode protection (4–6 digit PIN) and optional biometric authentication (Fingerprint / Face ID).
FR-SET-006	The system **shall** allow configuration of recurring expense/income reminder notifications.
FR-SET-007	The system **shall** provide budget alert notifications: warn at 80% usage and alert at 100% usage for any active plan category.
FR-SET-008	The system **shall** allow the user to set a monthly savings target percentage.

FR-SET-009	The system **shall** provide data management options: Create Backup, Restore Backup, Clear All Data.
FR-SET-010	The system **shall** allow optional Google Drive and Dropbox cloud synchronization configuration.
FR-SET-011	The system **shall** provide a 'Copy Purchase ID' function for in-app purchase verification.
FR-SET-012	The system **shall** allow configuring the Money Plan analysis lookback period (default 6 months; range 1–24 months).

3.9 Backup & Synchronization (FR-BAK)

FR-BAK-001	The system **shall** create local encrypted backups in SQLite format, exportable as a .mb file (Moneyora backup).
FR-BAK-002	The system **shall** support automatic scheduled local backups (Daily / Weekly — user configurable).
FR-BAK-003	The system **shall** optionally integrate with Google Drive for cloud backup when internet is available.
FR-BAK-004	The system **shall** optionally integrate with Dropbox for cloud backup when internet is available.
FR-BAK-005	The system **shall** support cross-device data restore from any valid Moneyora backup file.
FR-BAK-006	The system **shall** display a backup reminder notification if no backup has been created in 7 days.

4. EXTERNAL INTERFACE REQUIREMENTS

4.1 User Interface Requirements

- Material Design 3 guidelines (Android) / Cupertino guidelines (iOS) with custom green branding
- Minimum touch target size: 48 × 48 dp for all interactive elements
- Home screen: donut chart (center), category icons arranged around perimeter, Balance bar, + / – FABs
- Side drawers: Categories panel, Accounts panel, Currencies panel, Settings panel
- Expense/Income entry: custom numeric keypad, date selector, category row, account selector, note field
- Support for screen widths 360 dp to 420 dp (portrait primary; landscape optional)
- Consistent color language: Green for income, Red for expense, Teal for transfers and AI features
- Accessibility: minimum 4.5:1 contrast ratio, support for system font scaling

4.2 Hardware Interfaces

Component	Usage
Camera	Required for receipt scanning; accessed via Flutter camera plugin
Biometric Sensor	Fingerprint reader / Face ID for optional authentication
Local Storage	For encrypted SQLite database and receipt images
Network Adapter	Optional — for cloud sync and AI API calls when available

4.3 Software Interfaces

Package / Service	Purpose
sqflite + sqflite_sqlcipher	Local SQLite database with AES-256 encryption
google_mlkit_text_recognition	Offline OCR for receipt scanning (ML Kit v2)
fl_chart	Donut charts, bar charts, and line trend charts
flutter_local_notifications	Budget alerts and recurring expense reminders
image_picker / camera	Camera capture and gallery selection for receipts
share_plus / path_provider	File export (CSV, PDF) and filesystem access
http / dio	Optional: AI API calls and exchange rate updates
google_sign_in / googleapis	Optional: Google Drive cloud backup integration
flutter_dropbox	Optional: Dropbox cloud backup integration
local_auth	Biometric / PIN authentication

4.4 Communication Interfaces

Protocol	Usage
HTTPS / TLS 1.3	All outbound API calls when internet is used
REST API	Exchange rate updates; AI budget plan enhancement
OAuth 2.0	Google Drive and Dropbox authentication
Local Filesystem	Backup file read/write; receipt image storage

5. NON-FUNCTIONAL REQUIREMENTS

5.1 Performance Requirements

ID	Metric	Target
NFR-PER-001	App cold-start launch time	< 2 seconds
NFR-PER-002	Expense entry from home screen (taps)	≤ 3 taps to entry screen
NFR-PER-003	Receipt scan processing (OCR + parse)	< 5 seconds for standard receipt
NFR-PER-004	Money Plan generation (6 months data)	< 3 seconds
NFR-PER-005	Chart rendering on home screen	< 1 second
NFR-PER-006	Database queries (10,000 transactions)	< 100 ms per query
NFR-PER-007	Backup creation (full data)	< 10 seconds for typical dataset

5.2 Security Requirements

- **NFR-SEC-001**: Database shall be encrypted using AES-256 via SQLCipher at all times
- **NFR-SEC-002**: Receipt images shall be stored in encrypted app-private storage
- **NFR-SEC-003**: Optional PIN protection: 4–6 digit, with 5-attempt lockout and exponential backoff
- **NFR-SEC-004**: Optional biometric authentication (Fingerprint / Face ID)
- **NFR-SEC-005**: No financial data shall be transmitted to any server without explicit user action
- **NFR-SEC-006**: Cloud backups shall be encrypted before upload
- **NFR-SEC-007**: App shall not request unnecessary device permissions

5.3 Reliability & Availability

- **NFR-REL-001**: 100% core functionality available offline — zero internet dependency for basic use
- **NFR-REL-002**: Zero data loss on unexpected app crashes — all transactions written atomically
- **NFR-REL-003**: Auto-save: expense entries are persisted immediately on confirmation
- **NFR-REL-004**: Backup reminder if no backup has been created in 7 days
- **NFR-REL-005**: Database migration support for all future app version upgrades

5.4 Maintainability

- **NFR-MNT-001**: Clean Architecture pattern — strict separation: Presentation, Domain, Data layers
- **NFR-MNT-002**: Minimum 75% unit test coverage for Domain layer (business logic)
- **NFR-MNT-003**: All public APIs documented with Dart doc comments
- **NFR-MNT-004**: Comprehensive crash logging (Firebase Crashlytics or equivalent)
- **NFR-MNT-005**: Database schema versioned — migration scripts for every schema change

5.5 Portability

- **NFR-PRT-001**: Single Flutter codebase targeting both iOS and Android
- **NFR-PRT-002**: Responsive layout supporting screen widths 360–420 dp
- **NFR-PRT-003**: Localization framework (flutter_localizations) integrated from the start
- **NFR-PRT-004**: Backup files (.sb format) portable across iOS and Android devices

5.6 Scalability

Resource	Scale Target
Transactions	Up to 100,000 transactions without performance degradation
Categories	Up to 50 custom categories (in addition to 18 defaults)
Accounts	Up to 20 accounts
Receipt Images	Configurable max storage (default 500 MB); old images auto-archived
Budget Plans	Up to 20 saved named plans
Keyword Dictionary	Unlimited keyword-to-category mappings

6. DATA MODEL

6.1 Entity Relationship Overview

The following entities form the core data model. All tables are stored in a single SQLCipher-encrypted SQLite database file on the device.

```
User ■■< Account (1:N)
User ■■< Category (1:N)
User ■■< MoneyPlan (1:N)
User ■■< ReceiptScan (1:N)
Account ■■< Transaction (1:N)
Category ■■< Transaction (1:N)
Category ■■< PlanAllocation (1:N)
MoneyPlan ■■< PlanAllocation (1:N)
Transaction >■■< ReceiptItem (via receipt_scan_id, N:1)
ReceiptScan ■■< ReceiptItem (1:N)
Category ■■< KeywordDictionary (1:N)
Transaction ■■< RecurringRule (1:1, optional)
```

6.2 Core Table Definitions

■ Table: users

id	INTEGER PRIMARY KEY
passcode_hash	TEXT (nullable)
biometric_enabled	INTEGER DEFAULT 0
theme	TEXT DEFAULT 'light'
language	TEXT DEFAULT 'en'
currency	TEXT DEFAULT 'LKR'
first_day_week	INTEGER DEFAULT 0 (0=Sun)
first_day_month	INTEGER DEFAULT 1
savings_target_pct	REAL DEFAULT 0.0
plan_analysis_months	INTEGER DEFAULT 6
created_at	TEXT (ISO 8601)

■ Table: accounts

id	INTEGER PRIMARY KEY AUTOINCREMENT
user_id	INTEGER REFERENCES users(id)
name	TEXT NOT NULL
icon	TEXT NOT NULL

currency	TEXT DEFAULT 'LKR'
initial_balance	REAL DEFAULT 0.0
current_balance	REAL DEFAULT 0.0
initial_balance_date	TEXT
include_in_total	INTEGER DEFAULT 1
is_archived	INTEGER DEFAULT 0
created_at	TEXT

■ Table: categories

id	INTEGER PRIMARY KEY AUTOINCREMENT
user_id	INTEGER REFERENCES users(id)
name	TEXT NOT NULL
icon	TEXT NOT NULL
color	TEXT NOT NULL
type	TEXT CHECK(type IN ('expense','income'))
parent_id	INTEGER REFERENCES categories(id) (nullable)
is_default	INTEGER DEFAULT 0
sort_order	INTEGER DEFAULT 0

■ Table: transactions

id	INTEGER PRIMARY KEY AUTOINCREMENT
account_id	INTEGER REFERENCES accounts(id)
category_id	INTEGER REFERENCES categories(id)
amount	REAL NOT NULL
type	TEXT CHECK(type IN ('expense','income','transfer'))
date	TEXT NOT NULL (ISO 8601 date)
time	TEXT (HH:MM)
note	TEXT (nullable)
receipt_image_path	TEXT (nullable)
is_recurring	INTEGER DEFAULT 0
recurring_rule_id	INTEGER (nullable)
created_at	TEXT
updated_at	TEXT

■ Table: money_plans

id	INTEGER PRIMARY KEY AUTOINCREMENT
user_id	INTEGER REFERENCES users(id)
name	TEXT NOT NULL
period_type	TEXT (day/week/month/year/custom)
start_date	TEXT NOT NULL
end_date	TEXT NOT NULL
total_budget	REAL
is_active	INTEGER DEFAULT 0
created_at	TEXT

■ Table: plan_allocations

id	INTEGER PRIMARY KEY AUTOINCREMENT
plan_id	INTEGER REFERENCES money_plans(id)
category_id	INTEGER REFERENCES categories(id)
allocated_amount	REAL NOT NULL
spent_amount	REAL DEFAULT 0.0
confidence_level	TEXT (High/Medium/Low)
is_user_modified	INTEGER DEFAULT 0
notes	TEXT (nullable)

■ Table: receipt_scans

id	INTEGER PRIMARY KEY AUTOINCREMENT
user_id	INTEGER REFERENCES users(id)
image_path	TEXT NOT NULL
merchant_name	TEXT (nullable)
receipt_date	TEXT (nullable)
total_amount	REAL (nullable)
tax_amount	REAL (nullable)
confidence_score	INTEGER 0-100
status	TEXT (pending/confirmed/rejected)
created_at	TEXT

■ Table: receipt_items

id	INTEGER PRIMARY KEY AUTOINCREMENT
receipt_scan_id	INTEGER REFERENCES receipt_scans(id)
name	TEXT NOT NULL
quantity	REAL DEFAULT 1.0
unit_price	REAL
total_price	REAL NOT NULL
suggested_category_id	INTEGER (nullable)
confirmed_category_id	INTEGER REFERENCES categories(id) (nullable)
confidence_score	INTEGER 0-100

■ Table: keyword_dictionary

id	INTEGER PRIMARY KEY AUTOINCREMENT
keyword	TEXT NOT NULL
category_id	INTEGER REFERENCES categories(id)
match_type	TEXT (exact/contains/startswith)
priority	INTEGER DEFAULT 5
usage_count	INTEGER DEFAULT 0
is_user_defined	INTEGER DEFAULT 0

7. AI FEATURE SPECIFICATIONS

7.1 Money Plan Generation Algorithm

The algorithm runs in three phases:

■ Phase 1 — Data Collection & Classification

- Fetch all transactions from the configured lookback period (default 6 months)
- Group transactions by category
- For each category, compute: count, mean, median, standard deviation, min, max
- Classify as Fixed if coefficient of variation (CV = std/mean) < 0.15
- Classify as Seasonal if FFT analysis reveals periodic spikes aligned to monthly/quarterly cycles
- All remaining categories classified as Variable

■ Phase 2 — Allocation Calculation

```
For FIXED categories:
  allocation = average(last 3 occurrences)

For VARIABLE categories:
  w_recent = 0.60  (last 3 months)
  w_older  = 0.40  (months 4-6+)
  allocation = weighted_moving_average(amounts, w_recent, w_older)
  if seasonal_spike_detected(category, plan_period):
      allocation *= seasonal_multiplier
  trend = linear_regression_slope(category_amounts_over_time)
  if trend > 0: allocation *= 1.08  # 8% buffer
  if trend < 0: allocation *= 0.95  # reduce 5%

confidence = HIGH   if data_points >= 10 and CV < 0.25
confidence = MEDIUM if data_points >= 4  and CV < 0.50
confidence = LOW    otherwise
```

■ Phase 3 — Budget Constraint Application & Output

- If user specifies total budget: normalize all allocations proportionally to sum = total_budget
- If no constraint: sum all allocations = suggested_total
- Compute daily_allowance per category = allocation / number_of_days_in_period
- Return: plan object with per-category {allocation, confidence, daily_allowance, type}

7.2 Receipt Scanning Pipeline

Pipeline Step	Detail
Step 1 — Image Capture	Camera or gallery → Flutter image_picker → raw image bytes
Step 2 — Preprocessing	Deskew (Hough transform), contrast enhancement (CLAHE), binarization (Otsu), noise reduction (median filter), perspective correction
Step 3 — OCR	Google ML Kit Text Recognition v2 (offline) → raw text blocks with bounding boxes

Step 4 — Text Parsing	Regex patterns + heuristics → extract {merchant, date, items[], total, tax}
Step 5 — Item Extraction	Identify product lines: name + qty + unit_price + total_price per line
Step 6 — Categorization	Layer 1: keyword_dictionary lookup; Layer 2: user history match; Layer 3: merchant context bias
Step 7 — Confidence Scoring	Each item gets 0-100 confidence; receipt gets overall score = mean(item scores)
Step 8 — User Review	Display Review & Confirm screen; user edits, merges, discards items
Step 9 — Transaction Creation	Create expense record per confirmed item; link to receipt_scan record
Step 10 — Learning Update	For each user-corrected categorization, insert/update keyword_dictionary

7.3 AI Integration Strategy

AI Layer	Capability	Connectivity
On-Device (Always)	Google ML Kit OCR, statistical budget calculation, keyword matching	Offline — no internet needed
Optional Cloud AI	OpenAI GPT or Google Gemini for enhanced plan suggestions and item description understanding	Requires internet; graceful fallback to on-device
Exchange Rates	Free REST API (e.g. exchangerate.host) for multi-currency conversion	Requires internet; cached for 24h offline use

8. DEVELOPMENT ROADMAP

8.1 Sprint Plan

Sprint	Timeline	Deliverables
Sprint 1	Week 1–2	Project setup, Flutter scaffolding, Clean Architecture structure, SQLite schema, DB helper classes
Sprint 2	Week 3–4	Expense & Income entry screens, custom keypad, account selector, category picker, transaction list
Sprint 3	Week 5	Account management CRUD, transfers, multi-currency support
Sprint 4	Week 6	Home screen donut chart, date filters, analytics screens, income vs. expense comparison
Sprint 5	Week 7–8	Money Plan Generator: data analysis engine, UI wizard, plan display, real-time tracking overlay
Sprint 6	Week 9–10	Receipt Scanner: camera integration, ML Kit OCR, text parser, categorizer, Review screen
Sprint 7	Week 11	Settings, passcode/biometric auth, recurring reminders, notifications, dark theme
Sprint 8	Week 12	Backup/restore, CSV/PDF export, cloud sync (Google Drive/Dropbox), data migration
Sprint 9	Week 13	Unit testing (domain layer ≥75%), integration tests, UI polish, performance optimization
Sprint 10	Week 14	Beta testing, bug fixes, app store preparation, user manual, final documentation

8.2 Milestones

Milestone	Target Date	Success Criteria
M1 — Foundation	End Week 2	Project compiles; DB schema created; navigation working
M2 — Core Tracking	End Week 5	Full expense/income/account/transfer flow working
M3 — Analytics	End Week 6	Charts and filtering fully functional
M4 — AI Budget	End Week 8	Money Plan Generator producing plans from real data
M5 — Receipt AI	End Week 10	Receipt scanning categorizing items with >70% accuracy
M6 — Production Ready	End Week 14	All features complete, tested, and app-store ready

9. APPENDICES

Appendix A Full Category Reference

Category	Type	Typical Transactions
Bills	Expense	Utility bills, rent, loan payments
Car	Expense	Fuel, maintenance, parking, insurance
Clothes	Expense	Clothing, shoes, accessories
Communications	Expense	Mobile data, internet, phone bills
Eating Out	Expense	Restaurants, cafes, takeaway food
Entertainment	Expense	Movies, events, concerts, streaming
Food	Expense	Groceries, supermarket, food items
Gifts	Expense	Presents for others
Health	Expense	Medicine, doctor, pharmacy, fitness
House	Expense	Home maintenance, furniture, repairs
Pets	Expense	Pet food, vet, grooming
Sports	Expense	Gym, sports equipment, activities
Taxi	Expense	Uber, PickMe, tuk-tuk, cab fare
Toiletry	Expense	Personal hygiene, beauty products
Transport	Expense	Bus, train, fuel, public transport
Deposits	Income	Bank deposits, investment returns
Salary	Income	Monthly salary, wages, freelance pay
Savings	Income	Money moved into savings

Appendix B Technology Stack Summary

Layer	Technology	Purpose
Framework	Flutter 3.x (Dart 3.0+)	Cross-platform iOS & Android
Architecture	Clean Architecture (Feature-first)	Maintainability, testability
State Management	Riverpod 2.x	Reactive, scalable
Local Database	sqflite + SQLCipher	Offline-first with encryption
Charts	fl_chart	Donut, bar, line charts

OCR	google_mlkit_text_recognition	Offline receipt scanning
AI (optional)	OpenAI API / Gemini API	Enhanced budget suggestions
Auth	local_auth	Biometric + PIN
Notifications	flutter_local_notifications	Budget alerts, reminders
Export	pdf (dart) + csv	Data portability
Cloud Backup	googleapis + flutter_dropbox	Optional sync
Image	image_picker + camera	Receipt capture
Testing	flutter_test + mockito	Unit + widget tests
Crash Logging	firebase_crashlytics	Production error tracking

Appendix C Risk Register

ID	Risk Description	Likelihood	Impact	Mitigation
R1	OCR accuracy low on poor-quality receipts	Medium	High	Image preprocessing pipeline; low-confidence flagging for manual review
R2	AI API costs exceed budget	Low	Medium	Use free tiers (Gemini free); fallback to on-device algorithm
R3	ML Kit OCR not available on some devices	Low	High	Test on min SDK (API 26); fallback to manual entry prompt
R4	SQLCipher performance on large datasets	Low	Medium	Index key columns; benchmark at 50,000 transactions early
R5	App size exceeds 80 MB with ML models	Medium	Medium	Use ML Kit dynamic feature delivery; defer model download
R6	Insufficient transaction history for AI planning	High	Medium	Minimum 1 month data; show Low confidence; provide manual-entry plan